Azure Data Factory to Power BI

End-to-End Sales Analytics | Screenshot Documentation and Validation

Project overview

This hands-on project ingests sales data from Azure Blob Storage through Azure Data Factory (ADF) into Azure SQL Database, loads a dimensional warehouse with T-SQL stored procedures, and uses a Power BI semantic model to support an interactive sales report.

Architecture: CSV file → Azure Blob Storage → ADF Copy activity → staging.Sales → SQL stored procedures → FactSales and dimensions → Power BI semantic model → interactive report.

Verified project results

Fact rows	Unique sales	Quantity	Sales total
10	**10**	**18**	**$3,780**

Evidence index

01 Azure Blob Storage: source location

02 ADF pipeline: activity orchestration

03 ADF execution: successful run

04 SQL staging: loaded CSV rows

05 SQL fact validation: counts and uniqueness

06 Power BI semantic model: star schema

07 DAX measures: reusable business calculations

08 Power BI report: completed visuals

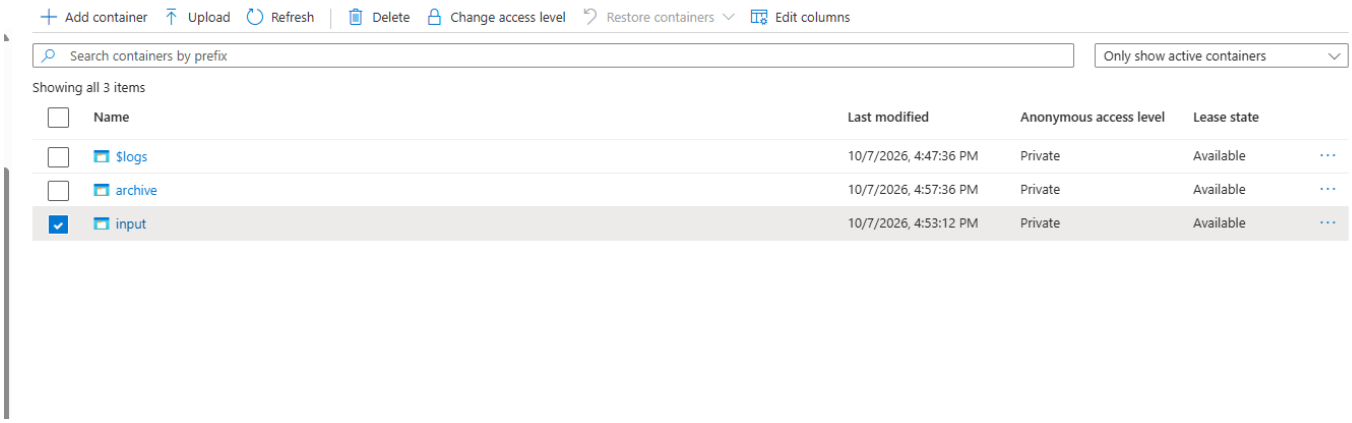
09 Date slicer: cross-filtering test

Scope note: The pipeline and report were successfully demonstrated using a 10-row practice dataset. Automated incremental processing, comprehensive failure handling, and file archiving were not fully implemented or validated.

Screenshot 1 of 9

Azure Blob Storage - Sales CSV source

Purpose: Source data storage



What this screenshot shows

Shows the Azure Storage area used for the sales CSV input. The source file is stored in the input container for ADF ingestion.

Why this step matters

Establishes the cloud-based landing zone for the source file and the starting point of the pipeline.

Evidence and interpretation

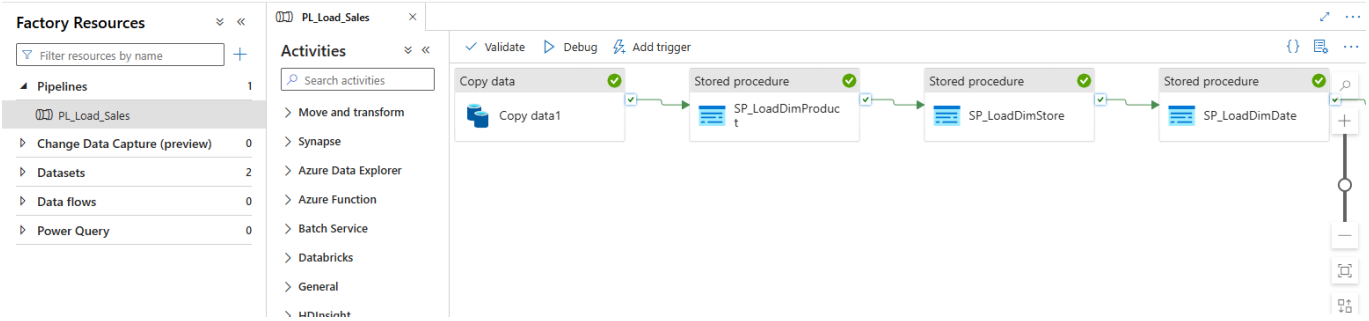
The storage containers are visible; this screenshot alone does not prove the CSV content or a completed pipeline run.

GitHub image: screenshots/01-blob-storage.png

Screenshot 2 of 9

Azure Data Factory - PL_Load_Sales

Purpose: Pipeline orchestration



What this screenshot shows

Shows the PL_Load_Sales pipeline with a Copy Data activity and linked SQL stored procedure activities for dimension and fact loading.

Why this step matters

Demonstrates how ingestion and SQL warehouse-loading steps are coordinated using success dependencies.

Evidence and interpretation

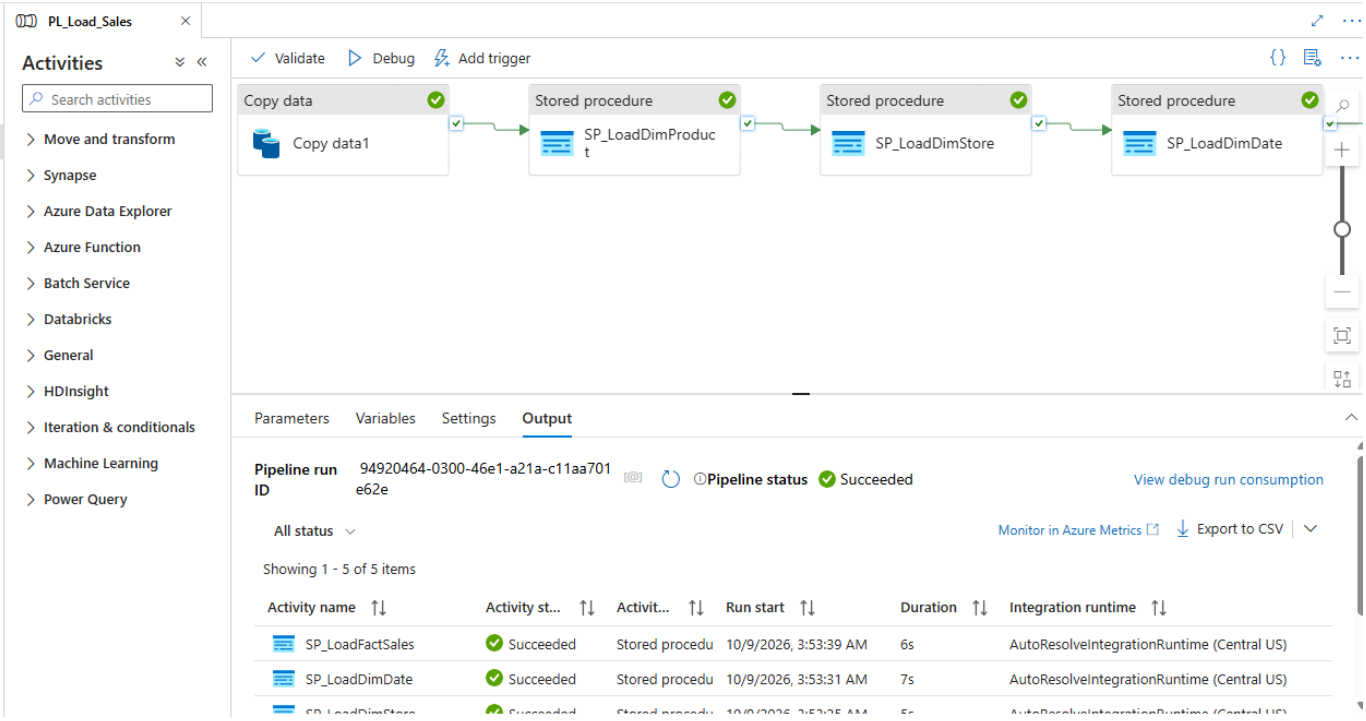
The screenshot captures part of the activity chain; the completed fact-load step was verified separately.

GitHub image: screenshots/02-adf-pipeline.png

Screenshot 3 of 9

ADF Pipeline - Successful Execution

Purpose: Pipeline execution validation



What this screenshot shows

Shows a successful ADF pipeline execution with activity-level results, including stored procedure activities.

Why this step matters

Confirms the orchestrated workflow was executed and its monitored activities succeeded in the test run.

Evidence and interpretation

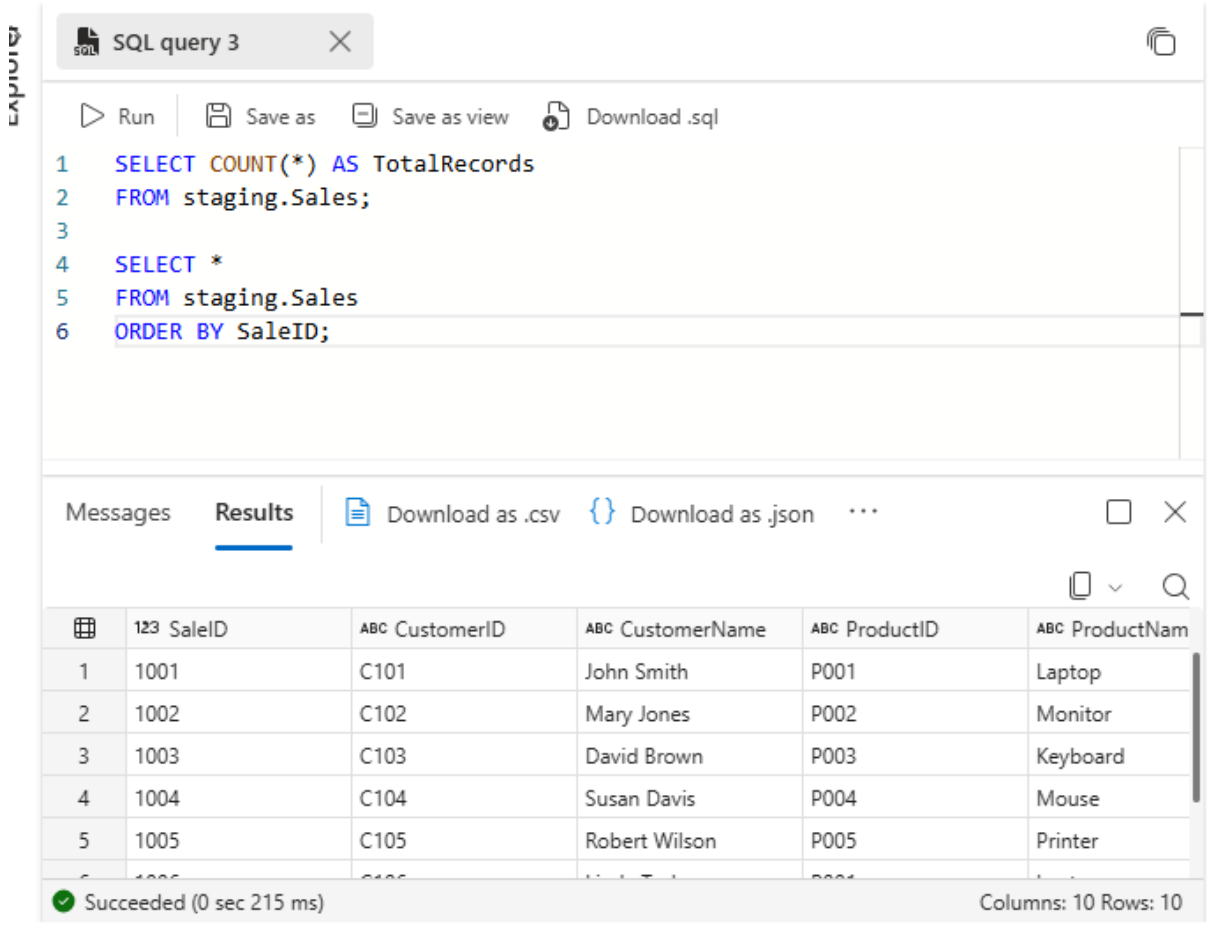
Successful execution is evidence of the tested run, not a guarantee of ongoing scheduled execution. The daily trigger was left stopped.

GitHub image: screenshots/03-pipeline-success.png

Screenshot 4 of 9

Azure SQL - staging.Sales Records

Purpose: Staging data inspection



What this screenshot shows

Shows a SELECT query against staging.Sales and the loaded source records.

Why this step matters

Confirms the CSV data reached the Azure SQL staging layer and could be inspected before dimensional loading.

Evidence and interpretation

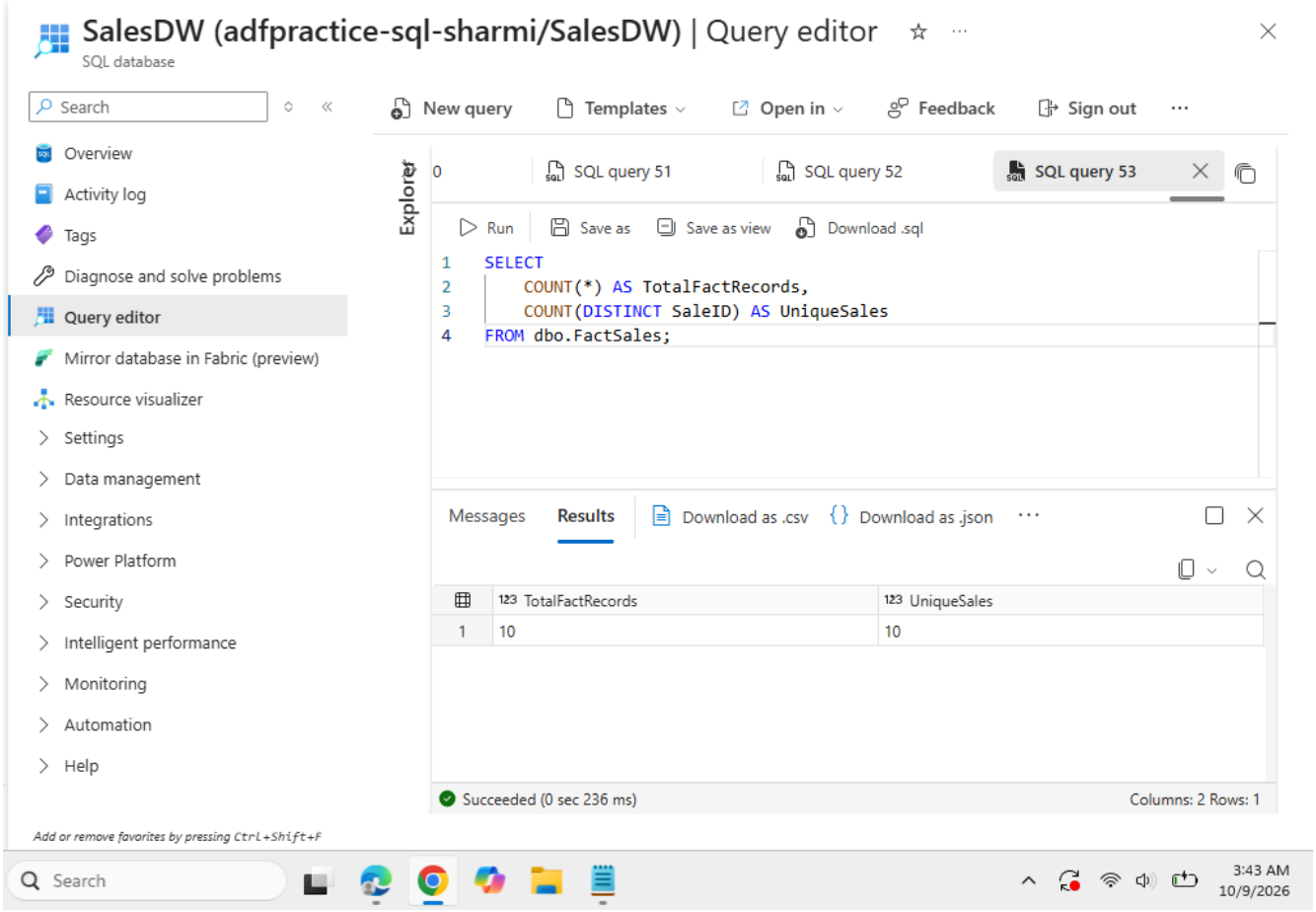
Ten staging rows were verified.

GitHub image: screenshots/04-sql-staging.png

Screenshot 5 of 9

Azure SQL - FactSales Validation

Purpose: Warehouse validation



What this screenshot shows

Shows a SQL validation query using COUNT(*) and COUNT(DISTINCT SaleID) on dbo.FactSales.

Why this step matters

Checks that the fact table contains the expected number of records without duplicate SaleID values.

Evidence and interpretation

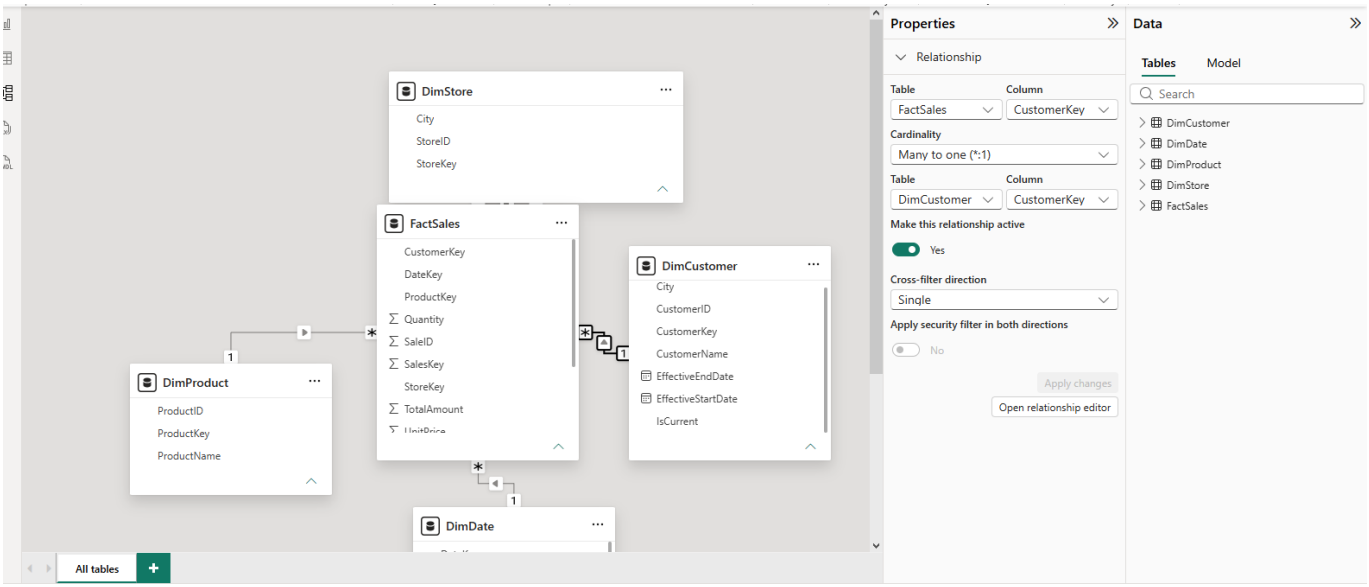
This screenshot directly shows 10 fact records and 10 unique sales. Total quantity 18 and sales $3,780 were confirmed by separate project queries.

GitHub image: screenshots/05-sql-validation.png

Screenshot 6 of 9

Power BI - Star-Schema Semantic Model

Purpose: Relationship modeling



What this screenshot shows

Shows the Power BI Model view connecting FactSales to DimCustomer, DimProduct, DimStore, and DimDate.

Why this step matters

Provides a reusable analytical layer where dimension filters flow to fact-table measures.

Evidence and interpretation

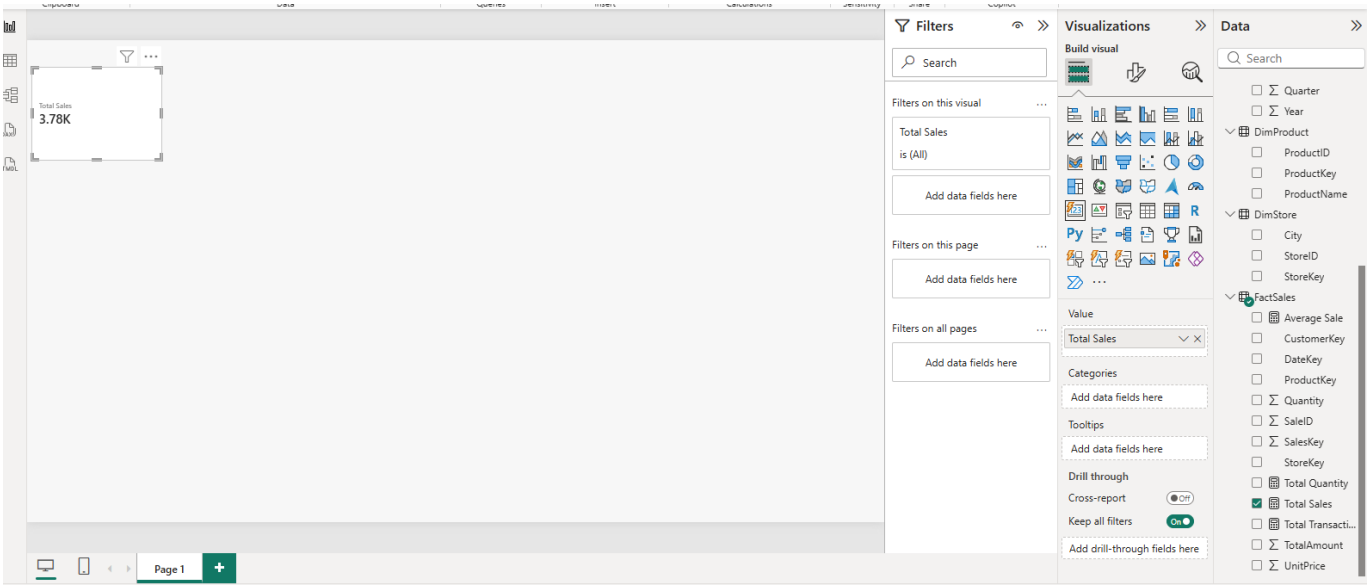
The FactSales-to-DimCustomer relationship was corrected to active many-to-one with single-direction filtering.

GitHub image: screenshots/06-semantic-model.png

Screenshot 7 of 9

Power BI - DAX Measures

Purpose: Business metric definitions



What this screenshot shows

Shows the Power BI report and the measure fields created under FactSales.

Why this step matters

Centralizes calculations used by the report: Total Sales, Total Quantity, Total Transactions, and Average Sale.

Evidence and interpretation

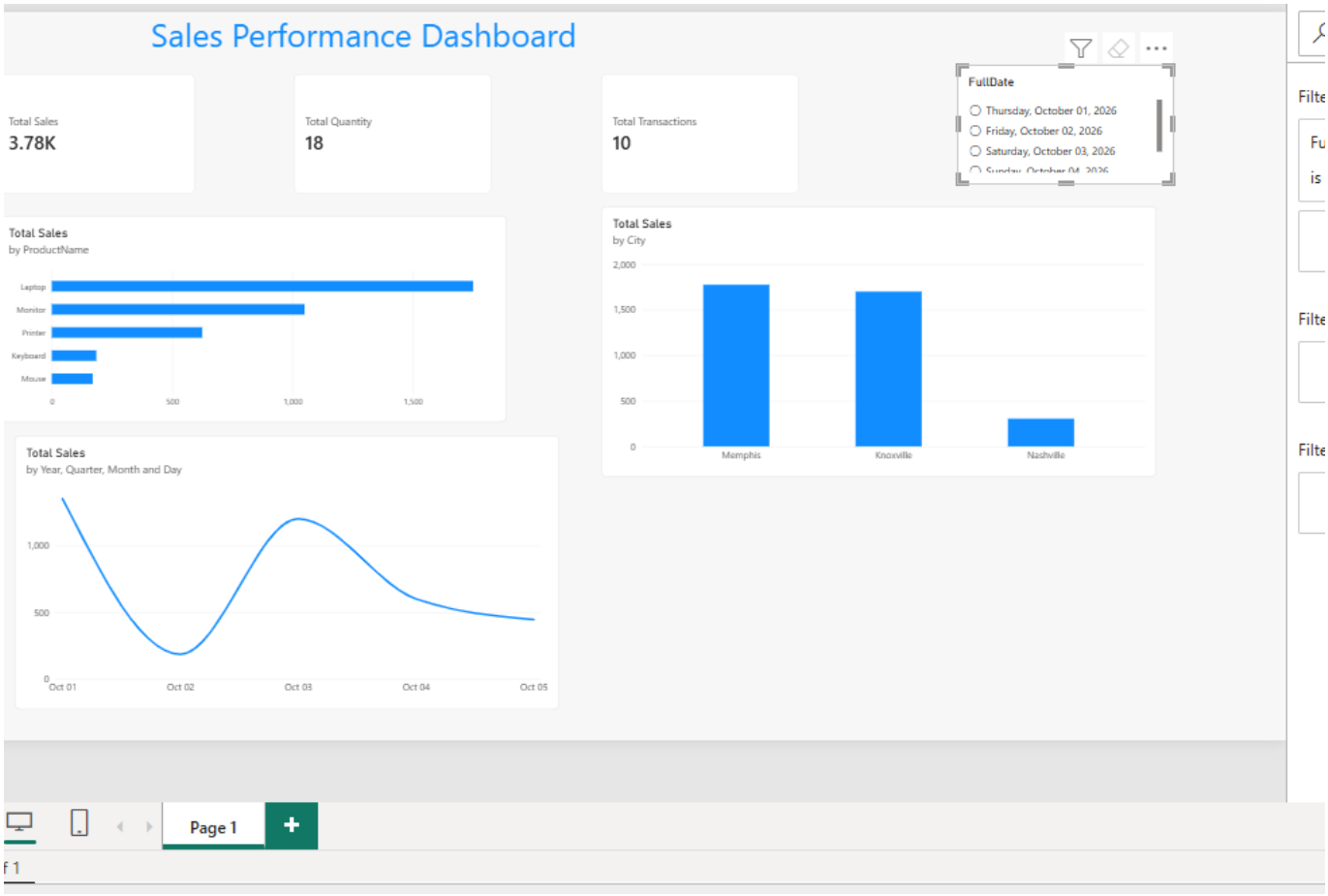
Formulas: SUM(TotalAmount); SUM(Quantity); DISTINCTCOUNT(SaleID); DIVIDE(Total Sales, Total Transactions, 0).

GitHub image: screenshots/07-dax-measures.png

Screenshot 8 of 9

Power BI - Sales Performance Report

Purpose: Business intelligence presentation



What this screenshot shows

Shows the completed report with three KPI cards, sales by product, sales by city, a sales trend, and the FullDate slicer.

Why this step matters

Transforms warehouse records into a readable, interactive analytical report.

Evidence and interpretation

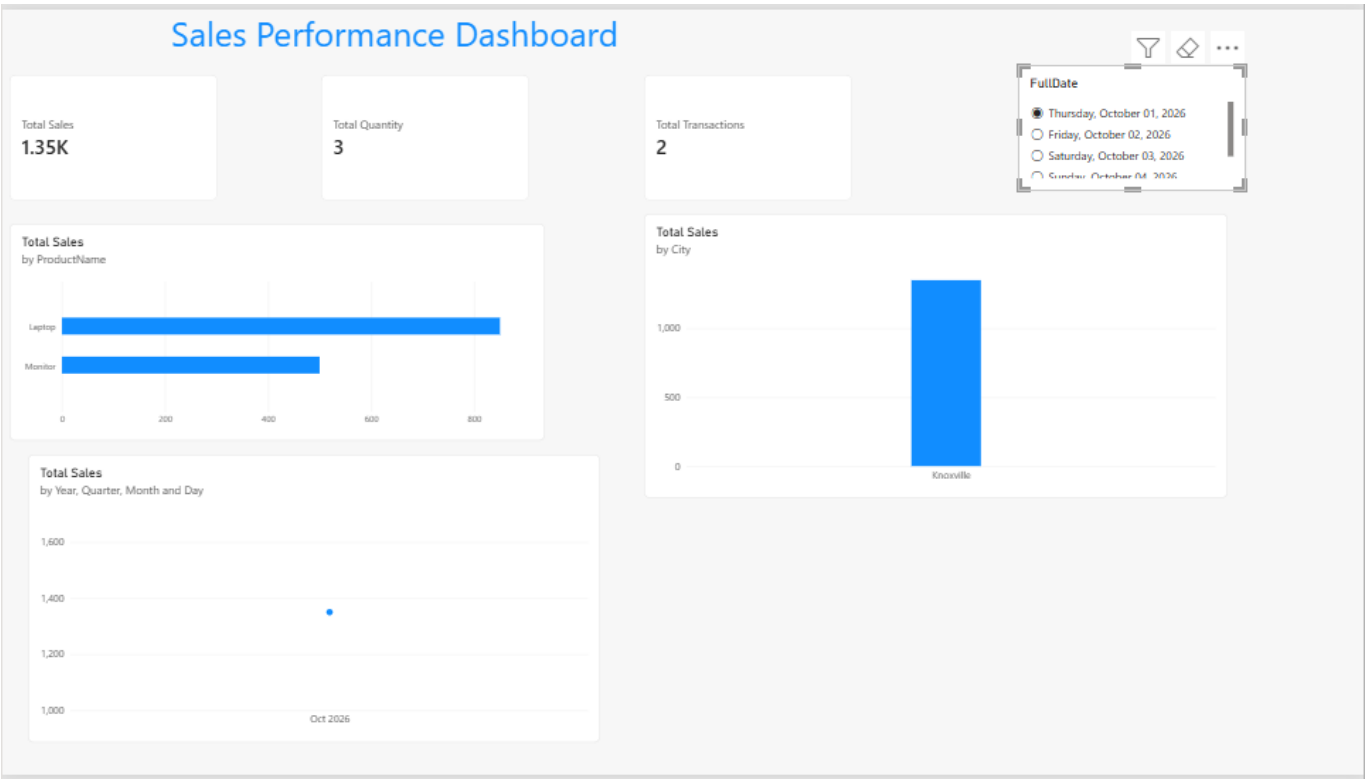
Unfiltered report totals: $3,780 in sales, 18 units, and 10 transactions.

GitHub image: screenshots/08-sales-report.png

Screenshot 9 of 9

Power BI - Date Slicer Validation

Purpose: Interactive filtering test



What this screenshot shows

Shows the report filtered to October 1, 2026 using the FullDate list slicer.

Why this step matters

Verifies the date dimension relationship and that slicer selections filter measures and report visuals.

Evidence and interpretation

Filtered results: $1,350 sales, 3 units, and 2 transactions; product, city, and trend visuals also respond.

GitHub image: screenshots/09-interactive-filter.png

Implementation summary and GitHub notes

Completed: Blob source storage, ADF Copy activity, SQL staging, stored procedure-based dimensional/fact loading, a successful test pipeline run, star-schema relationships, DAX measures, report visuals, and validated interactive date filtering.

Project scope: A small demonstration dataset (10 transactions) was used to validate the end-to-end flow. The ADF daily trigger was configured but intentionally stopped. Fully automated incremental processing, production monitoring and alerting, comprehensive error handling, and source file archiving are future improvements.

Recommended repository layout: README.md, screenshots/ (nine numbered PNGs), sql/ (sanitized SQL scripts), adf/ (sanitized pipeline definitions if available), powerbi/ (reviewed PBIX if appropriate), docs/ (this PDF).

Security review before public upload: Inspect screenshots, SQL scripts, ADF JSON, and the PBIX for connection strings, secrets, access keys, account identifiers, personal information, or imported sensitive data. Remove or redact these before publishing.

Suggested GitHub project description

Built an end-to-end sales analytics solution using Azure Blob Storage, Azure Data Factory, Azure SQL Database, dimensional modeling, and Power BI. Implemented and validated data ingestion, warehouse loading, DAX business measures, and interactive reporting.